

Práctica Experimental Unidad III

Figueroa Bryan

Romero Bryam

Velez Ricardo

1 Introduction

El desarrollo de aplicaciones web modernas exige comprender tanto los mecanismos internos que gestionan el estado de una petición HTTP como las decisiones arquitectónicas que determinan la escalabilidad, el mantenimiento y el rendimiento de un sistema. La presente investigación aborda, con rigor científico-bibliográfico y respaldo de fuentes académicas de acceso abierto (IEEE Xplore, ACM Digital Library, arXiv, MDPI, DOAJ y revistas indexadas), los cuatro temas que conforman la Unidad 3: los objetos integrados para la gestión de estado, el acceso a datos desde aplicaciones web, los patrones de arquitectura de software y el diseño de arquitecturas web escalables. El documento sigue el formato de citación IEEE, referenciando cada fuente utilizada al final del trabajo.

2 Objetos integrados y gestión del estado en aplicaciones web

2.1 El objeto Request

El objeto Request encapsula la información que el cliente envía al servidor dentro de una petición HTTP, incluyendo parámetros, cabeceras, cookies y el cuerpo del mensaje. En plataformas basadas en Java, el contenedor web transforma cada petición entrante en un objeto `HttpServletRequest` que es entregado al componente encargado de procesarla [1], [2]. Este objeto constituye el punto de entrada de datos hacia la lógica de negocio y es la base sobre la cual se construyen mecanismos de enrutamiento, validación y control de acceso.

2.2 El objeto Response

El objeto Response representa la información que el servidor devuelve al cliente tras procesar la solicitud; incluye el cuerpo del contenido, el código de estado HTTP y las cabeceras de salida, entre ellas las utilizadas para el envío de cookies mediante métodos como `addCookie()` [2]. Este objeto es responsable de convertir el resultado del procesamiento interno del servidor en un mensaje HTTP comprensible para el navegador, cerrando así el ciclo petición-respuesta que caracteriza a la arquitectura cliente-servidor [1].

2.3 El objeto Session

El objeto Session permite mantener el estado de un usuario a través de múltiples peticiones HTTP, las cuales son, por naturaleza, independientes entre sí. La interfaz `HttpSession` posibilita almacenar y recuperar atributos asociados a un cliente específico durante el tiempo que dure su interacción con la aplicación [2]. En arquitecturas orientadas a objetos de sesión, como WebObjects, el servidor crea un objeto de sesión en la primera petición del usuario y restaura su estado en cada ciclo posterior, garantizando la persistencia de la información entre transacciones sucesivas [3]. La correcta gestión de sesiones es determinante para la escalabilidad, ya que la afinidad de servidor asociada al almacenamiento local de sesiones puede limitar el balanceo de carga en entornos distribuidos [4].

2.4 Los objetos Application y Page

El objeto Application almacena información compartida por todas las sesiones activas dentro de un mismo directorio virtual o contexto de despliegue, sirviendo como espacio de datos común a nivel de toda la aplicación [1]. Por su parte, el objeto Page (o el ámbito de página) delimita el ciclo de vida más corto dentro del modelo de objetos integrados, existiendo únicamente durante el procesamiento de una única petición-respuesta. La combinación jerárquica de estos ámbitos —petición, sesión y aplicación— define el modelo clásico de gestión de estado en aplicaciones web dinámicas, y su diseño influye directamente en decisiones posteriores de escalabilidad, como el uso de componentes sin estado (*stateless*) para facilitar el despliegue en granjas de servidores [4].

3 Accediendo a datos desde aplicaciones web

3.1 Tecnologías de acceso a datos en aplicaciones web

El acceso a datos desde una aplicación web puede realizarse mediante distintas tecnologías, entre las que destacan la conectividad nativa a través de controladores estándar (JDBC en el ecosistema Java) y las capas de abstracción orientadas a objetos como los proveedores de la Java Persistence API (JPA). JDBC es una interfaz de programación (API) que define cómo una aplicación interactúa con una base de datos mediante sentencias SQL, siendo independiente del motor de base de datos utilizado, aunque su uso requiere un controlador específico para cada fabricante [5]. La elección entre acceso directo y capas de abstracción constituye una decisión arquitectónica que impacta el rendimiento, la mantenibilidad y la portabilidad del sistema.

3.2 Consultas SQL desde aplicaciones web

La ejecución de consultas SQL desde el código de la aplicación es el mecanismo más directo de acceso a datos y ofrece control explícito sobre el rendimiento de cada operación. Estudios comparativos entre el uso de JDBC puro y frameworks

de mapeo objeto-relacional han demostrado que la ejecución de sentencias SQL controladas manualmente (por ejemplo mediante `PreparedStatement`) puede superar en rendimiento a las herramientas ORM, aunque a costa de mayor esfuerzo de codificación y menor abstracción del modelo de dominio [6], [7]. Este hallazgo resulta relevante al momento de seleccionar la estrategia de persistencia según los requisitos no funcionales del proyecto.

3.3 Procedimientos almacenados y funciones de BD

Los procedimientos almacenados y las funciones de base de datos son bloques de código SQL precompilados y almacenados en el motor de datos, que permiten encapsular lógica de negocio cerca de los datos, reducir el tráfico de red entre la aplicación y el servidor de base de datos, y reutilizar rutinas complejas de manera centralizada. Su uso está estrechamente relacionado con las decisiones de arquitectura por capas, ya que definen dónde reside la lógica de acceso y transformación de datos: en la capa de aplicación o en la capa de persistencia [5]. Este equilibrio entre lógica embebida en la base de datos y lógica gestionada por el framework de acceso a datos es también objeto de análisis en la literatura sobre patrones de persistencia empresarial [8].

3.4 ORM y mapeo objeto-relacional

El mapeo objeto-relacional (ORM, por sus siglas en inglés) es una técnica que permite convertir datos entre el modelo de objetos utilizado por un lenguaje de programación orientado a objetos y el modelo relacional de una base de datos, resolviendo el denominado “desajuste de impedancia objeto-relacional” [5], [7]. Entre los frameworks más estudiados en la literatura académica se encuentran Hibernate, EclipseLink, OpenJPA y DataNucleus, cuyo desempeño ha sido comparado en estudios experimentales que evalúan tiempos de ejecución, configuración y facilidad de uso [8]. Los resultados de estas investigaciones muestran que, si bien el ORM introduce una sobrecarga de rendimiento frente al SQL directo, sus beneficios en productividad, mantenibilidad y abstracción del modelo de dominio lo convierten en una alternativa ampliamente adoptada en el desarrollo empresarial [7], [9].

3.5 Principios de arquitectura de software

La arquitectura de software se define como el conjunto de decisiones estructurales significativas que determinan la organización de un sistema, sus componentes, las relaciones entre ellos y los principios que guían su diseño y evolución [10]. Según el enfoque contemporáneo basado en atributos de calidad, la arquitectura debe diseñarse priorizando requisitos no funcionales —como el rendimiento, la seguridad, la mantenibilidad, la disponibilidad y la escalabilidad— por encima de la funcionalidad exclusivamente [11], [12]. El proceso de diseño arquitectónico es iterativo: se estiman los atributos de calidad, se evalúa el diseño frente a dichos requisitos y, en caso de no cumplirse, se procede a una fase de transformación arquitectónica hasta lograr conformidad [12].

3.6 Arquitectura por capas

La arquitectura por capas (*layered architecture*) organiza el sistema en niveles jerárquicos con responsabilidades específicas, típicamente presentación, lógica de negocio y acceso a datos, favoreciendo la separación de responsabilidades y facilitando el mantenimiento del sistema [9], [13]. No obstante, este estilo puede introducir sobrecarga de rendimiento debido al paso de datos entre capas y tiende a favorecer un escalado vertical antes que horizontal, lo que limita su flexibilidad en escenarios de alta demanda [14].

3.7 Arquitecturas en pizarra, dirigida por eventos y peer-to-peer

La arquitectura en pizarra (*blackboard*) se basa en un espacio de datos compartido al cual acceden múltiples componentes especializados que contribuyen incrementalmente a la solución de un problema, siendo especialmente útil en sistemas donde no existe un algoritmo determinístico único. La arquitectura dirigida por eventos (*event-driven architecture*) organiza el flujo del sistema en torno a la generación, detección y consumo asíncrono de eventos, lo que permite un alto rendimiento en el procesamiento de grandes volúmenes de información, aunque presenta desafíos relacionados con la consistencia eventual de los datos [14]. Por su parte, la arquitectura peer-to-peer (P2P) distribuye responsabilidades de manera simétrica entre los nodos participantes, sin depender de un servidor central, lo que favorece la resiliencia pero complejiza la coordinación y el descubrimiento de servicios.

3.8 Arquitectura Orientada a Servicios (SOA) y Microservicios

La Arquitectura Orientada a Servicios (SOA) descompone la aplicación en servicios débilmente acoplados que se comunican mediante protocolos estandarizados como SOAP o HTTP/XML, favoreciendo la reutilización de funcionalidad y la interoperabilidad entre sistemas heterogéneos [9], [15]. Su evolución histórica está ligada a tecnologías como XML-RPC y al surgimiento del *Enterprise Service Bus* (ESB) como mecanismo para gestionar la creciente complejidad de las integraciones [15]. La arquitectura de microservicios, considerada una evolución de SOA, propone la construcción de sistemas como conjuntos de servicios pequeños, independientemente desplegables y organizados en torno a capacidades de negocio específicas, comunicados mediante API ligeras [13], [16]. Estudios recientes destacan que, si bien los microservicios mejoran la escalabilidad, la tolerancia a fallos y la flexibilidad operativa —tal como lo evidencian casos de empresas como Netflix, Amazon y Uber—, también introducen retos significativos en materia de consistencia de datos, gestión de API y coordinación de transacciones distribuidas, para lo cual se recomiendan prácticas como el Diseño Guiado por el Dominio (DDD) y el patrón Saga [16]. En el ámbito de la salud digital, investigaciones aplicadas han comparado plataformas basadas en SOA y en microservicios para el despliegue de servicios en entornos de computación en la nube, evidenciando ventajas de los microservicios en escalabilidad selectiva

frente a la mayor rigidez de los buses de servicio en SOA [17].

4 Diseñando Arquitecturas Web Escalables

4.1 Escalabilidad en aplicaciones web

La escalabilidad es la capacidad de un sistema para mantener niveles aceptables de rendimiento y disponibilidad ante el incremento de la carga de trabajo, ya sea mediante el aumento de los recursos de un mismo servidor (escalado vertical) o mediante la incorporación de nuevos nodos (escalado horizontal). El uso de balanceo de carga permite distribuir las peticiones entrantes entre múltiples servidores web, maximizando el tiempo de respuesta y el aprovechamiento de recursos, aunque su eficacia depende de evitar la afinidad de servidor y de diseñar componentes sin estado (*stateless*) [4]. Patrones como el *Intercepting Filter* y el *Front Controller* contribuyen a centralizar el procesamiento de peticiones, facilitando la aplicación de mecanismos transversales de escalabilidad [4].

4.2 Patrones de caché en aplicaciones web

El almacenamiento en caché constituye una de las estrategias más efectivas para reducir la latencia y mejorar la escalabilidad de una aplicación web, al evitar el recálculo o la recuperación repetida de datos desde el origen. Estudios experimentales han demostrado que el caché en memoria (*in-memory*) puede reducir el tiempo de respuesta hasta en un 62,6 % respecto a escenarios sin caché, mientras que el caché basado en archivos y el caché de base de datos ofrecen mejoras de aproximadamente 36,6 % y 55,1 % respectivamente [18]. La literatura distingue entre el caché a nivel de aplicación —insertado manualmente en el código base y sujeto a errores por su naturaleza transversal— y el caché a nivel de infraestructura o de contenido, evaluándose criterios de reemplazo como *Least Recently Used* (LRU), cuyo rendimiento puede ser superado por estrategias conscientes de la distribución estadística de acceso (Zipf) en más de un 10 % de tasa de aciertos [19], [20]. Un estudio cualitativo con diez proyectos de software reales identificó patrones y buenas prácticas para el diseño, implementación y mantenimiento del caché a nivel de aplicación, resaltando su carácter de preocupación transversal (*crosscutting concern*) ajena a la lógica de negocio [21].

4.3 Documentación y evaluación de arquitecturas web

La documentación de la arquitectura de software formaliza las decisiones estructurales adoptadas y sus justificaciones, permitiendo su comunicación entre los interesados del proyecto y su evaluación posterior. El método de análisis de arquitecturas basado en atributos de calidad, desarrollado en el ámbito del *Software Engineering Institute* (SEI), propone priorizar los atributos de calidad relevantes para el sistema, diseñar estructuras que reflejen los compromisos (*trade-offs*) entre dichos atributos y, finalmente, evaluar el diseño resultante frente a los requisitos priorizados [11]. Las

Architecture Decision Records (ADR) se han consolidado como el formato estándar para documentar estas decisiones, registrando el contexto, la decisión adoptada, las alternativas consideradas y sus consecuencias, lo que evita la repetición de discusiones ya resueltas y facilita la evolución controlada del sistema [10].

Referencias

- [1] H. Yang, “Asp objects: Request, response, session and application,” <https://www.herongyang.com/VBScript/IIS-ASP-Object-Request-Response-Session-Application.html>, consultado en línea.
- [2] GeeksforGeeks, “Servlet: Request, response, and session,” <https://www.geeksforgeeks.org/advance-java/servlet-request-response-and-session/>, 2025.
- [3] Apple Inc., “Session management, webobjects developer guide,” https://developer.apple.com/library/archive/documentation/LegacyTechnologies/WebObjects/WebObjects_3.1/DevGuide/HowWOWorks/SessionMgt.html.
- [4] Guidance Share, “Application architecture guide – chapter 15: Web applications,” https://www.guidanceshare.com/wiki/Application_Architecture_Guide_-_Chapter_15_-_Web_Applications.
- [5] University of Business and Technology in Kosovo, “Performance analysis of jpa implementations,” <https://fileserv-az.core.ac.uk/download/268080806.pdf>, uBT Knowledge Center.
- [6] S. N. Bhatti, Z. H. Abro, and F. Rufabro, “Performance evaluation of java based object relational mapping tools,” 2013, dOAJ.
- [7] P. V. Zyl, D. G. Kourie, and A. Boake, “Comparing the performance of object databases and orm tools,” in *Proceedings of SAICSIT*.
- [8] “Comparing the performance of the object-relational mapping programming frameworks available in java,” *Journal of Computer Sciences Institute*, 2022.
- [9] S. Lazarenko, “Exploring common software architecture patterns: Monolithic, soa, microservices, and layered,” <https://staskoltsov.medium.com/exploring-common-software-architecture-patterns-monolithic-soa-microservices-and-layered-804e>, 2023.
- [10] Technology.org, “Software architecture guide: Principles, patterns & best practices,” <https://www.technology.org/how-and-why/software-architecture-guide/>, 2025.
- [11] “Architecting complex, long-lived scientific software,” <https://arxiv.org/pdf/2304.13797>, 2023, arXiv:2304.13797.
- [12] “Evaluation and classifying software architecture styles and quality attributes,” *American Journal of Basic and Applied Sciences (AJBAS)*, 2011.

- [13] GeeksforGeeks, “Software architectural patterns in system design,” <https://www.geeksforgeeks.org/system-design/design-patterns-architecture/>, 2025.
- [14] vFunction, “Enterprise software architecture patterns: The complete guide,” <https://vfunction.com/blog/enterprise-software-architecture-patterns/>, 2025.
- [15] The Open Group, “Soa microservices architecture,” <https://www.opengroup.org.cn/sites/default/files/2020-04/w169.pdf>, white Paper W169.
- [16] “Evolving software architectures from monolithic systems to resilient microservices: Best practices, challenges and future trends,” *International Journal of Advanced Computer Science and Applications (IJACSA)*, vol. 15, no. 9, 2024.
- [17] “Evaluating service-oriented and microservice architecture patterns to deploy ehealth applications in cloud computing environment,” *Applied Sciences*, vol. 11, no. 10, p. 4350, 2021.
- [18] “Reducing latency with caching: An in-depth analysis of effective caching strategies for modern web applications,” <https://www.researchgate.net/publication/385737667>, 2024, researchGate.
- [19] “A new class of web caching strategies for content delivery,” in *IEEE Conference Publication*. IEEE Xplore.
- [20] J. Mertz and I. Nunes, “Understanding application-level caching in web applications: A comprehensive introduction and survey of state-of-the-art approaches,” *ACM Computing Surveys*, vol. 50, no. 6, 2017.
- [21] —, “A qualitative study of application-level caching,” *IEEE Transactions on Software Engineering*.